

RI402

Dizajn kompajlera

dr.sc. Edin Pjanić

Pregled predavanja

- Leksička analiza
- Regularni jezici
- Regularni izrazi

Kompajler

- Leksička analiza
- Parsiranje
- Semantička analiza
- Generisanje koda

Leksička analiza

- Primjer programa:

```
if (i == j)
    z = 0;
else
    z = 1;
```

- Ulaz je niz znakova (karaktera):

```
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;\n
```

Tačnije, ulaz je: 9 105 102 32 40 105 32 61 61 32 106 41 10 9 9 122 32 61
32 48 59 10 9 101 108 115 101 10 9 9 122 32 61 32 49 59 10

- Cilj je razdvojiti ulazni string u podstringove (leksičke jedinice), ali ne samo to...

Tokeni

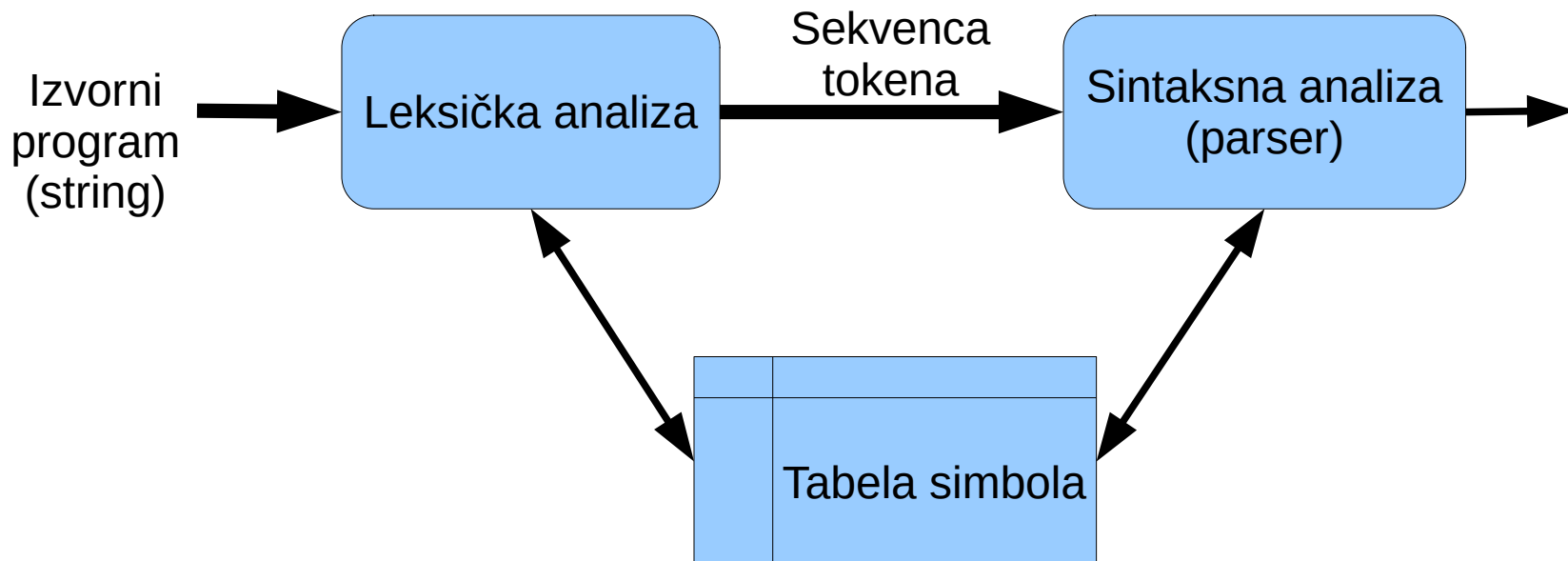
- Kompajler treba da klasifikuje različite elemente ulaznog stringa u skladu sa njihovom ulogom
 - tokeni
- Vrste (klase) tokena:
 - Prirodni (govorni) jezik
 - imenica, glagol, zamjenica, broj, pridjev, prilog, ...
 - Programski jezik
 - ključna riječ, identifikator, broj, (,), =, +, *, ==, ...

Tokeni

- Svaka klasa tokena korespondira sa nekim skupom stringova koji imaju istu ili srodnu ulogu.
- Primjeri za neke klase tokena:
 - Identifikator:
 - skup stringova koji sadrže slova i brojeve a počinje slovom (npr. `printf`, `temp`, `Var1`, ...)
 - Cijeli broj:
 - string cifara (npr. `67`, `747`, `0`, `000`, ...)
 - Ključne riječi:
 - `"if"` , `"else"` , `"do"` , `"while"`, ...
 - Praznine (whitespace):
 - sekvenca razmaka, novih redova ili tabova

Tokeni

- Leksička analiza:
 - Klasifikacija podstringova programa u skladu sa njihovom ulogom.
- Token - apstraktni simbol koji predstavlja vrstu leksičke jedinice:
 - $\langle \text{klasa tokena, atributi} \rangle$



Primjeri

- Ulaz je niz znakova (karaktera). Tokenizirati ulazni string!
- Primjer 1:

```
\tif (i == j)\n\t\ttz = 0;\n\telse\n\t\ttz = 1;\n
```

- Primjer 2:

```
x=0;\n\twhile (i < 10) {\n\ttx++;\n}
```

- Primjer 3:

```
printf("Ukupno: %d\n", vrijednost);
```


Leksički analizator: dizajn

- Korak 1:
 - Definirati konačan skup klasa tokena
 - Leksemi koji pripadaju istoj klasi imaju istu ili srodnu ulogu.
 - Broj klasa tokena zavisi od jezika.
- Korak 2:
 - Za svaku klasu tokena opisati koji stringovi im pripadaju

Leksički analizator: implementacija

- Implementacija LA mora :
 - Prepoznati podstringove koji korespondiraju sa tokenima
 - Leksem: sekvenca karaktera koji odgovaraju uzorku za neki token (konkretna instanca tokena)
 - Identifikovati klasu tokena za svaki leksem
Token: <klasa tokena, leksem>
- Obično se odbace tokeni koji nisu od interesa za parser
 - Prazna mjesta, komentari

Leksička analiza: primjeri problema kroz istoriju (zbog dizajna programskog jezika)

- FORTRAN: prazna mjesta se nisu razmatrala:
 - `VAR1` je isto kao i `VA R1`
 - `A X = AA + 5 5` je isto kao i `AX=AA+55`
- Petlja ima oblik `DO 10 I=1,N`
 - Razmotrimo izraze:

<code>DO 5 I = 1,25</code>	<code>D05I = 1,25</code>
<code>DO 5 I = 1.25</code>	<code>D05I = 1.25</code>
- Problem sa tokenizacijom: nemamo dovoljno informacija da bismo donijeli odluku o klasi tokena
- “Lookahead”

Leksička analiza: primjeri kroz istoriju

- PL/I: ključne riječi nisu rezervisane riječi

IF A THEN B = A; ELSE A = B

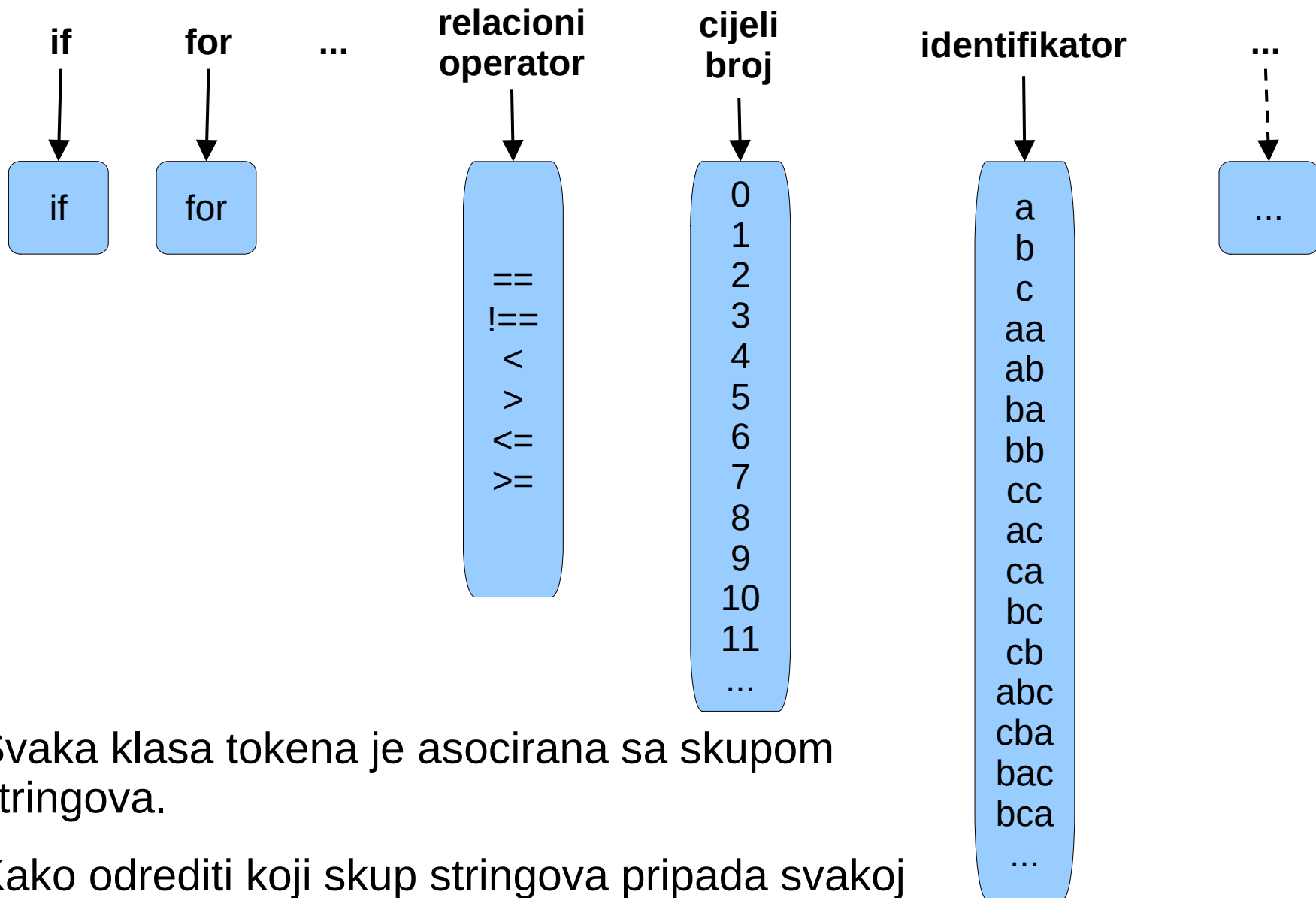
IF ELSE THEN THEN = ELSE; ELSE ELSE = THEN

- Problemi u slučaju C++:

Foo<Bar<Bazz>>

Foo<Bar<Bazz> >

- Leksičku strukturu programskog jezika čini skup klasa tokena.



- Svaka klasa tokena je asocirana sa skupom stringova.
- Kako odrediti koji skup stringova pripada svakoj klasi tokena?
- Rješenje: **Regularni jezici** (regular languages)

Jezik

- Definicija:

Neka je Σ skup znakova (alfabet).

Jezik nad skupom Σ je skup ograničenih nizova znakova uzetih iz skupa Σ . Označava se sa $L(\Sigma)$.

- Primjer:

- Neka je alfabet: $\Sigma = \{ 'a', 'b' \}$

- Jezik bi mogao biti:

- $L(\Sigma) = \{ „a”, „b”, „aa”, „aba”, „abba”, „baba”, „abab”, „abaaaab” \}$

- Problem: Na koji način bismo mogli opisati ovaj jezik (ili neki drugi)?

Regularni jezik

- Definicija za jezik:

Neka je Σ skup znakova (alfabet).

Jezik nad skupom Σ je skup ograničenih nizova znakova uzetih iz skupa Σ .

Označava se sa $L(\Sigma)$.

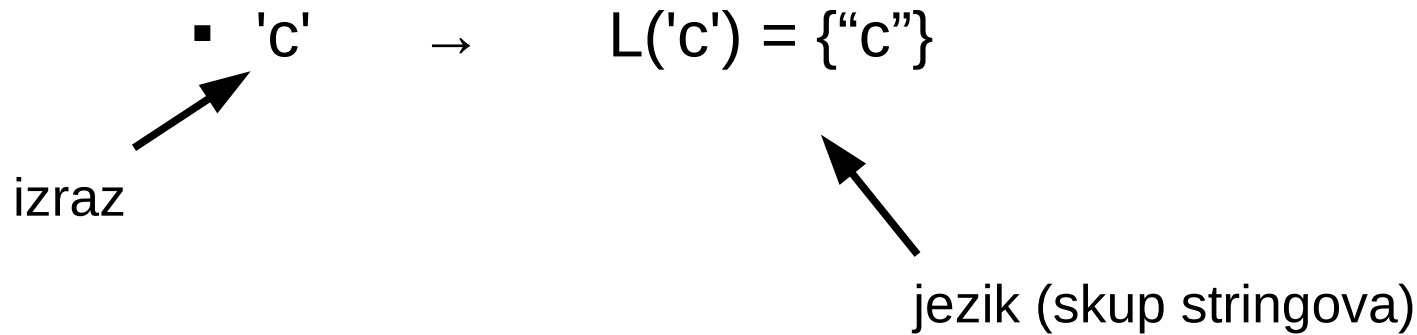
Regularni jezik je:

- Formalni jezik koji se može opisati korištenjem regularnih izraza (regular expressions).
- Svaki regularni izraz određuje jedan skup (regularni jezik).
- Praktično je to uzorak koji opisuje jedan skup stringova.

Regularni izrazi

Osnovni izrazi

- Jedan znak (karakter)



- Epsilon

▪ ε → $L(\varepsilon) = \{ "" \}$ prazan string (nije prazan skup)

$$L(\varepsilon) \neq \emptyset$$

'a'	$L('a') = \{ "a" \}$
'1'	$L('1') = \{ "1" \}$
'{'	$L('{') = \{ "{" \}$

Složeni regularni izrazi

A i B su regularni izrazi

- Unija

- $A + B : L(A+B) = \{a \mid a \in L(A)\} \cup \{b \mid b \in L(B)\}$

- Nadovezivanje

- $AB : L(AB) = \{ab \mid a \in L(A) \text{ i } b \in L(B)\}$

- Iteracija

- *Jednostavna*: $A^i = \underbrace{AA \dots A}_{i \text{ puta}} \quad A^0 = \varepsilon$

- *Klinijeva (Kleene)*: $A^* = \bigcup_{i \geq 0} A^i \quad \varepsilon + A + AA + AAA + \dots$

- *Klinijeva pozitivna*: $A^+ = \bigcup_{i \geq 1} A^i \quad A + AA + AAA + \dots$

Primjeri

- Neka je dat alfabet $\Sigma = \{0, 1\}$
- $1^* =$

$$U_{i \geq 0} 1^i = \varepsilon + 1 + 11 + 111 + 1111 + 11111 + \dots$$

$$L(1^*) = \{ "", "1", "11", "111", "1111", "11111", \dots \}$$

- tj. svi stringovi sa nula ili više jedinica

- $(0+1)1 =$
- $0^* + 1^* =$

Primjeri

Prilikom rada sa regularnim izrazima česta praksa je da se imenuju regularni izrazi i koriste u drugim regularnim izrazima. Npr.:

- Integer: string koji se sastoji od cifara
 - $\text{cifra} = '0' + '1' + '2' + '3' + '4' + '5' + '6' + '7' + '8' + '9'$
 - $\text{integer} = \text{cifra} \text{cifra}^*$
 - $\text{integer} = \text{cifra}^+$
- Praznina: razmak, tab ili novi red
 - $(' ' + '\t' + '\n')^+$

Primjeri

Identifikator: string koji se sastoji od slova i cifara, a počinje slovom

- cifra = '0' + '1' + '2' + '3' + '4' + '5' + '6' + '7' + '8' + '9'
- slovo = 'a' + 'b' + 'c' + 'd' + 'e' + 'f' + 'g' + 'h' + 'i' + 'j' + 'k' + 'l' + 'm' + 'n' + 'o' + 'p' + 'q' + 'r' + 's' + 't' + 'u' + 'v' + 'w' + 'x' + 'y' + 'z' + 'A' + 'B' + 'C' + 'D' + 'E' + 'F' + 'G' + 'H' + 'I' + 'J' + 'K' + 'L' + 'M' + 'N' + 'O' + 'P' + 'Q' + 'R' + 'S' + 'T' + 'U' + 'V' + 'W' + 'X' + 'Y' + 'Z'
- identifikator = slovo (slovo + cifra)*